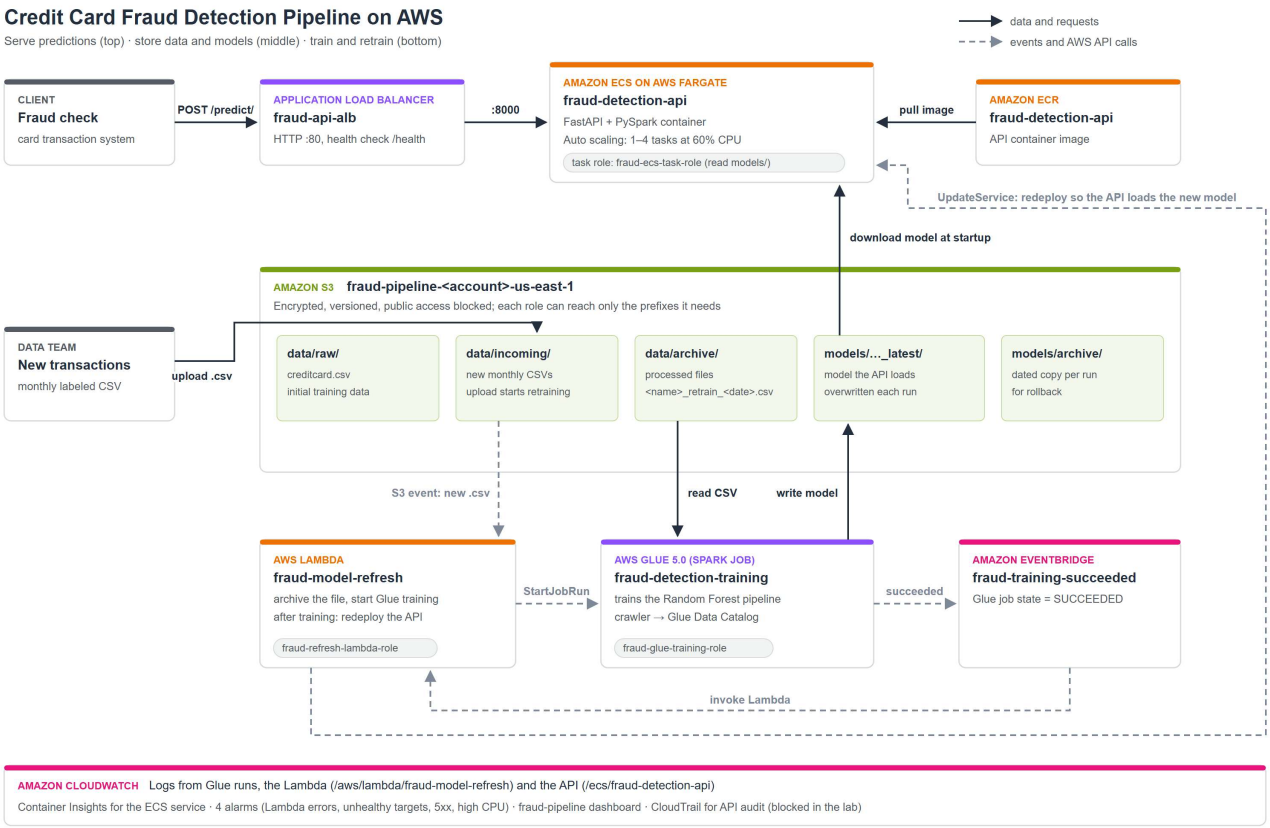
# Migrating a Credit Card Fraud Pipeline to AWS

## 1. Selecting Architecture for Workloads

### Selected AWS Services

| Pipeline component | AWS service(s) selected | Why does this service fit the project requirements? |
|---|---|---|
| Data and model storage | *Amazon S3* | *One durable, low-cost store for the raw dataset, new training data, and every trained model. Glue, Lambda, and ECS all read and write S3 natively. S3 event notifications trigger retraining when a new file lands. Default encryption, blocked public access, versioning, and least-privilege IAM policies keep data secure and earlier models remain recoverable.* |
| Model training job | *AWS Glue (Spark ETL job), Glue Data Catalog* | *Glue runs the existing PySpark code as a fully managed, serverless Spark job, so the team has no cluster to manage. It bills only while the job runs (about 4 minutes per training run here), so a monthly schedule carries no ongoing infrastructure cost. It integrates with S3, Lambda, EventBridge, and CloudWatch. A Glue crawler already catalogs the training data, the first step toward cataloging all of the team's datasets.* |
| Prediction API | *Amazon ECS on AWS Fargate, Application Load Balancer, Amazon ECR* | *Fargate runs the FastAPI container without managing servers or a Kubernetes cluster. ECS Service Auto Scaling grows the service from 1 to 4 tasks when average CPU passes 60%, and the load balancer spreads requests across healthy tasks. Container logs go to CloudWatch through the awslogs driver, and CloudTrail can record ECS API activity. An ECS task role gives the container read-only access to the model in S3, so no credentials are stored in the image.* |
| Retraining or refresh workflow | *AWS Lambda (S3 trigger), Amazon EventBridge* | *Lambda is serverless and charges per invocation, so it costs almost nothing for a monthly workload. A new .csv in data/incoming/ invokes it automatically. It archives the file and starts the Glue job through the Glue API. When training succeeds, an EventBridge rule invokes the function again to redeploy the API. So the new "latest" model goes into service with no manual steps.* |
| Monitoring or logs | *Amazon CloudWatch (Logs, Metrics, Alarms, Dashboard, Container Insights), AWS CloudTrail* | *CloudWatch collects logs and metrics from Glue runs, Lambda invocations, and the ECS service in one place. Alarms flag Lambda errors, unhealthy API tasks, server errors, and sustained high CPU, and a dashboard shows API traffic, latency, and task load. CloudTrail provides the audit trail of AWS API calls, such as who started a training job or changed an IAM role. It is part of the design but blocked in the Udacity lab account.* |



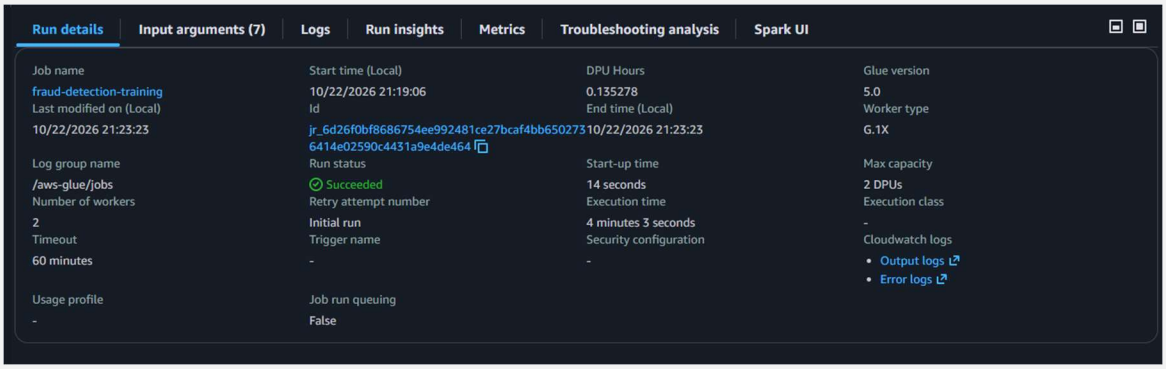

# 2. Migrating Workloads

## Modified Code Files

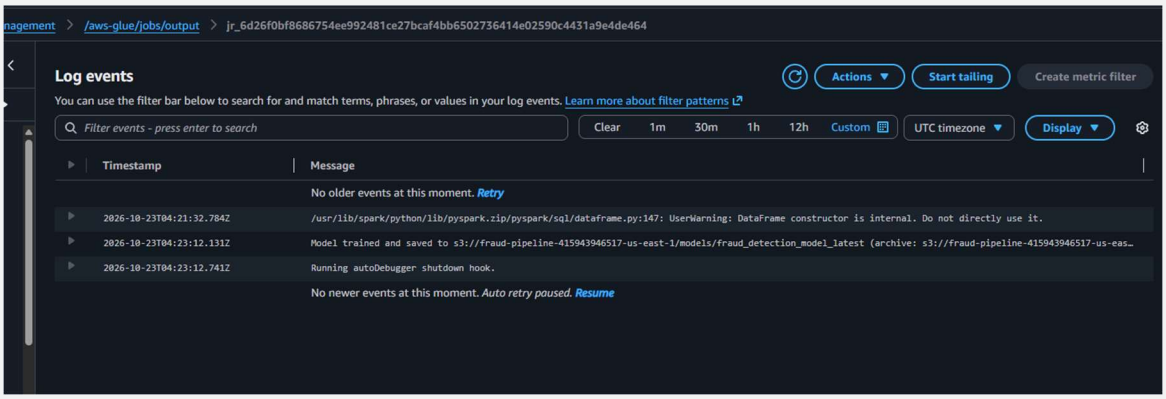
| File | Submitted ? | What changed? |
|---|---|---|
| fraud_detector_model_trainer.py | Yes | Converted to an AWS Glue job. The local SparkSession and __main__ block were replaced by a Glue entry point (GlueContext, Job) that reads JOB_NAME, DATA_URL, and MODEL_SAVE_PATH with getResolvedOptions. The local input file ../data/credit_card_transaction_data_labeled.csv became an S3 path, read as a DynamicFrame. MyTransform now takes a DynamicFrameCollection, converts it to a DataFrame for training, and returns a DynamicFrameCollection. The local save path ../fraud_detection_api/model/... was replaced by S3: the model is saved as models/fraud_detection_model_latest (what the API loads) plus a dated copy in models/archive/. Class is cast to an integer because the CSV quotes it. The try/except that printed and swallowed errors was |

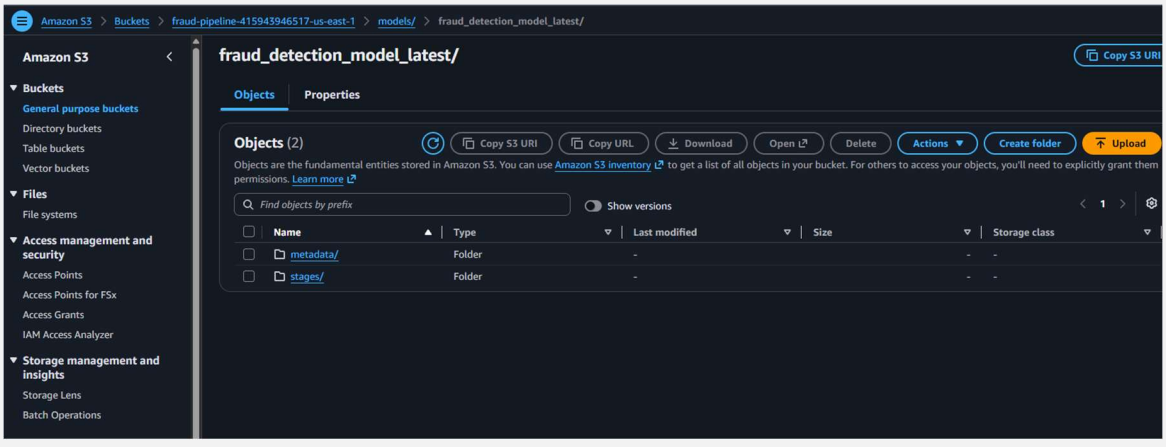
| | | |
|---|---|---|
| | | removed, so a failed run shows as Failed in Glue and CloudWatch. |
| fraud_detection_api/main.py | Yes | The local model path model/fraud_detection_model_latest was replaced by a download from S3 (MODEL_S3_BUCKET, MODEL_S3_PREFIX) to an intermediate local folder (LOCAL_MODEL_PATH, default /tmp), because Spark loads a saved model from a filesystem path. AWS credentials come from the ECS task role, not from code. The copy bundled in the image is used only when no bucket is set, for local development. Startup moved from the deprecated @app.on_event('startup') to a FastAPI lifespan handler. A GET /health endpoint was added for load balancer health checks, and an unused import was removed. |
| refresh_function.py or equivalent retraining workflow code | Yes | Rewritten as an AWS Lambda handler. The infinite loop polling a local folder (/Users/jonathandyer/...) every 10 seconds was replaced by an S3 ObjectCreated trigger on data/incoming/. The local os.rename became an S3 copy-and-delete into data/archive/, keeping the <name>_retrain_<date>.csv naming. Training no longer runs inside the function: Lambda cannot run Spark, so the function starts the Glue job through boto3 (start_job_run) on the archived file. A second trigger, an EventBridge rule for "Glue job succeeded", makes the same function force a new ECS deployment so the API loads the new model. The function's nested copy of the training code was removed because it is no longer imported, and its Pipfile now lists only boto3. |
| Deployment files, such as Dockerfile, requirements, or configuration notes | Yes | fraud_detection_api/Dockerfile: the base image changed from the retired apache/spark-py:v3.3.1 (Python 3.8) to python:3.14-slim with Java 21, which current PySpark needs. fraud_detection_api/requirements.txt: recreated with pinned versions and boto3 added for S3 access. fraud_detection_api/.dockerignore: new; keeps .env and other local files out of the image. fraud_detection_api/.env.example: new; documents the S3 and region settings, with credential slots for local development only. infra/config.sh, infra/deploy.sh, infra/teardown.sh: new; AWS CLI scripts that create, and delete, the S3 bucket, IAM roles, Glue job and crawler, ECR repository, ECS service, load balancer, auto scaling, Lambda triggers, and CloudWatch alarms and dashboard. .gitignore: keeps the 144 MB dataset, .env, and build files out of git. CloudFormation template is located in infra\cloudformation |

## Training Workload Evidence




**Caption:** This shows that the run was successful, it also shows the start and end times, duration, Glue version, worker type and DPU hours. This was started by the retraining Lambda.

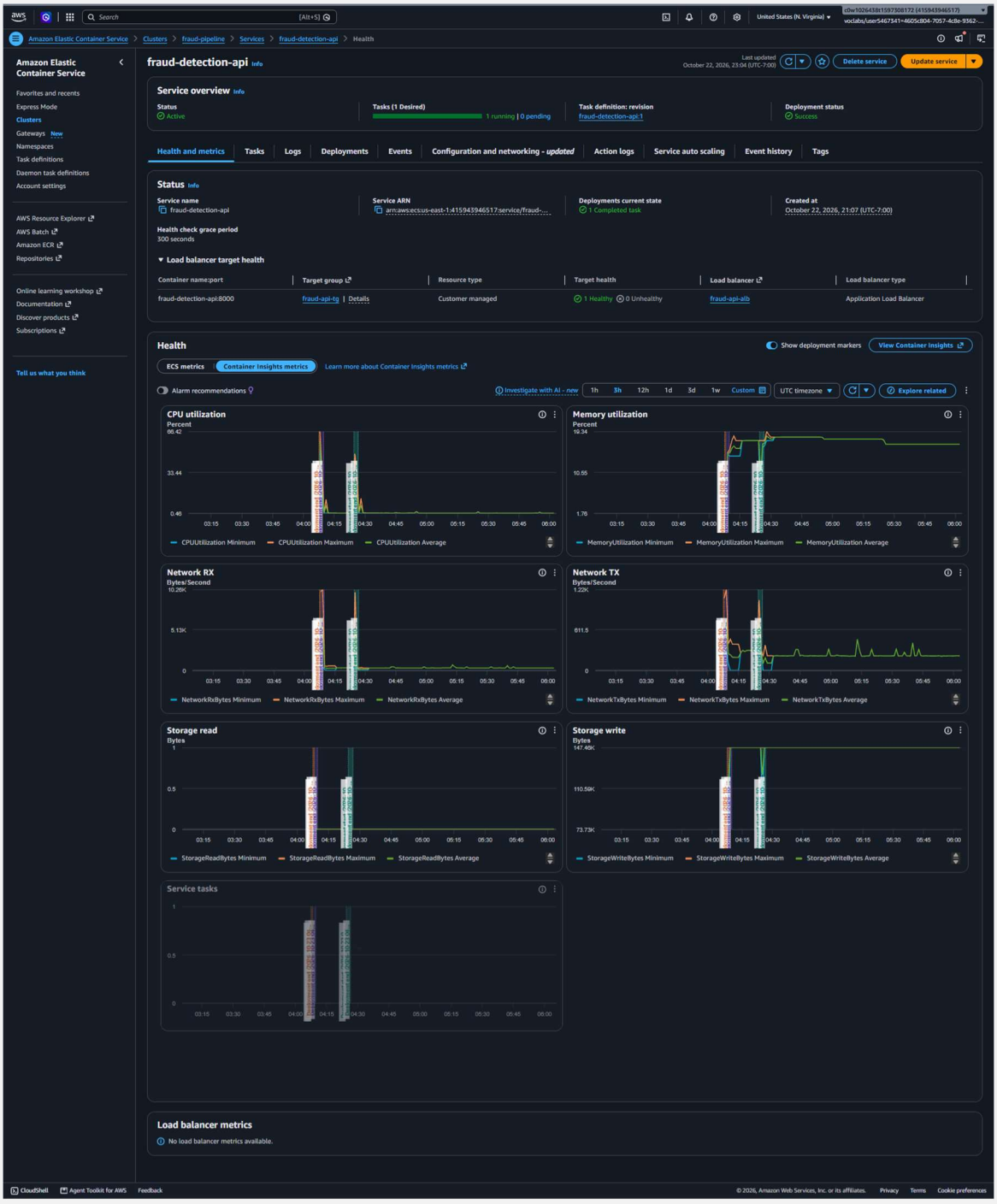



**Caption:** This log entry shows where in the s3 bucket the model is saved, the S3 file structure shows where

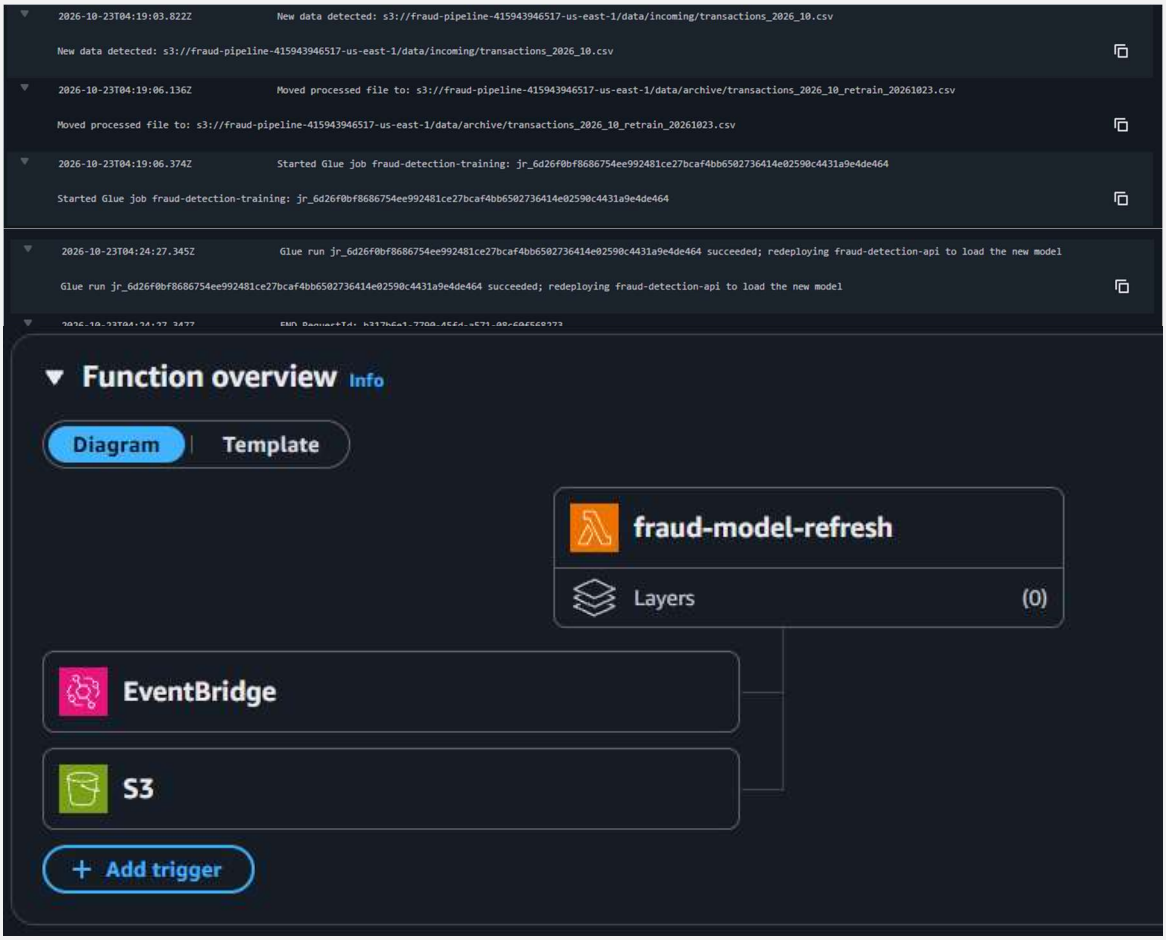
the artifacts are saved.

## Prediction API Evidence



**Caption:** The fraud-detection-api service on Amazon ECS (Fargate) in the fraud-pipeline cluster. Its status is Active, with 1 of 1 tasks running (0 pending), task definition fraud-detection-api:1, and deployment status Success. The Application Load Balancer fraud-api-alb reports the container fraud-detection-api:8000 as 1 Healthy, 0 Unhealthy, which means the API's /health endpoint is answering. The Container Insights graphs show the task's CPU, memory and network activity, including the redeployment after retraining (around 04:24 UTC).

## Retraining or Refresh Evidence



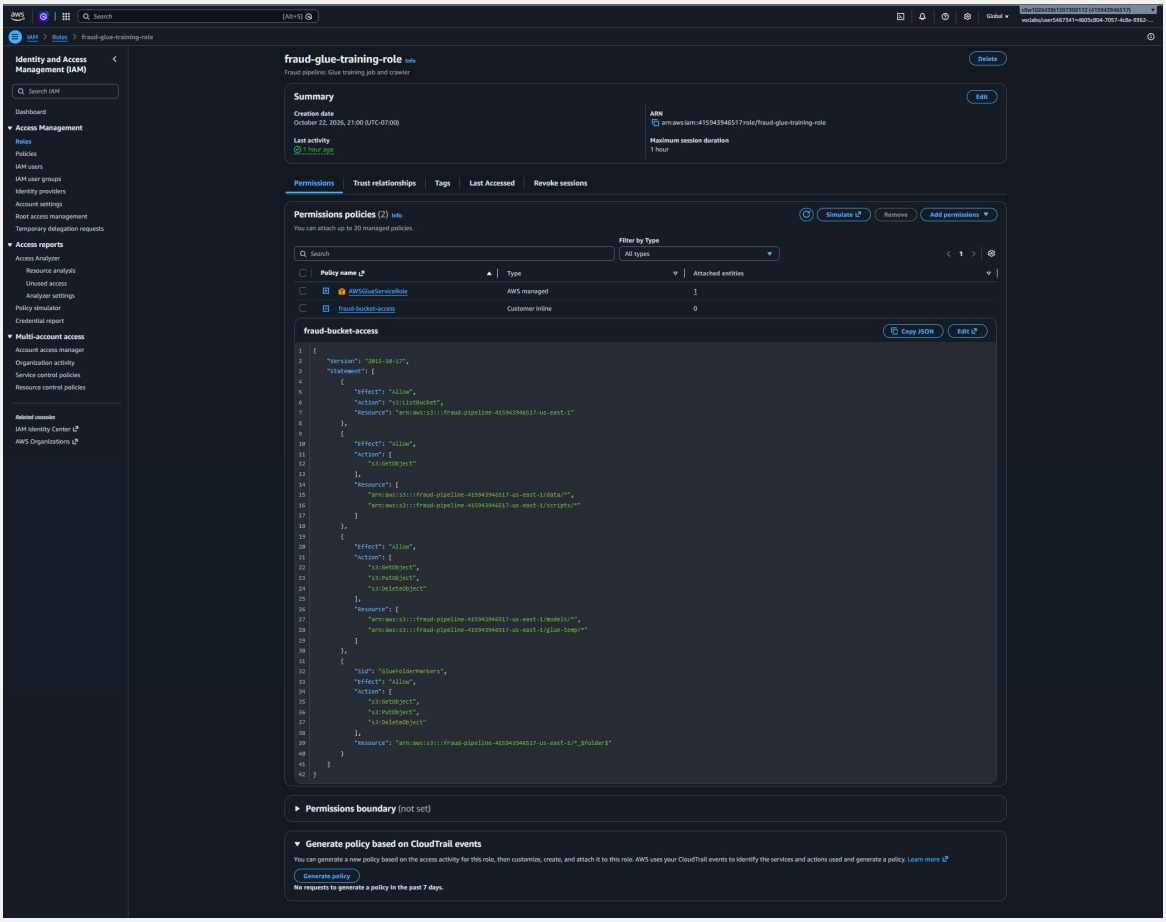



**Caption:** CloudWatch logs from the fraud-model-refresh AWS Lambda function. A new CSV in the S3 data/incoming/ prefix triggered the function, which archived the file and started AWS Glue training run jr_6d26f0bf…. When that run succeeded, an EventBridge rule invoked the function again, and it forced a new deployment of the fraud-detection-api service on Amazon ECS (Fargate), so the API reloads the retrained model. The fraud-model-refresh Lambda has two triggers. An S3 event fires when a new .csv lands in data/incoming/, and the function starts the Glue training job. An EventBridge rule fires when that Glue job succeeds, and the function redeploys the API so it loads the retrained model.

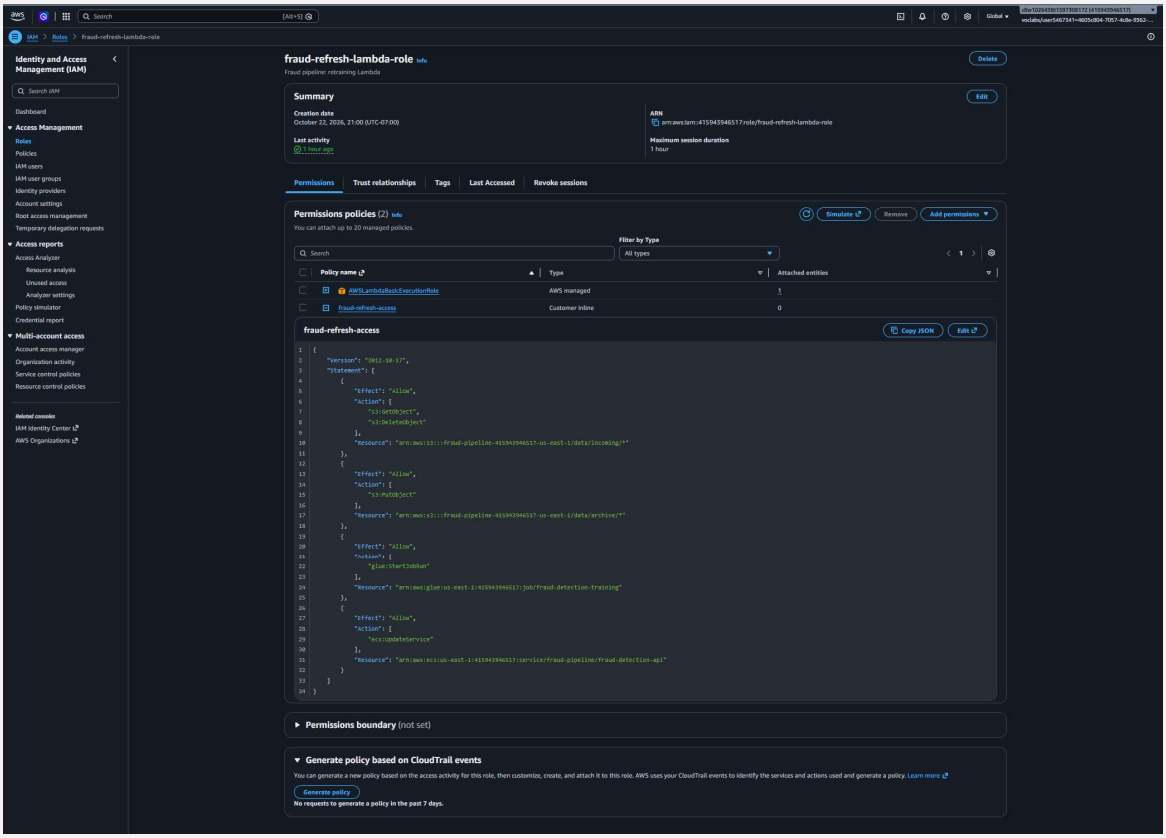
# 3. Determining New Architecture for Cloud Workloads

## IAM Roles and Service Permissions

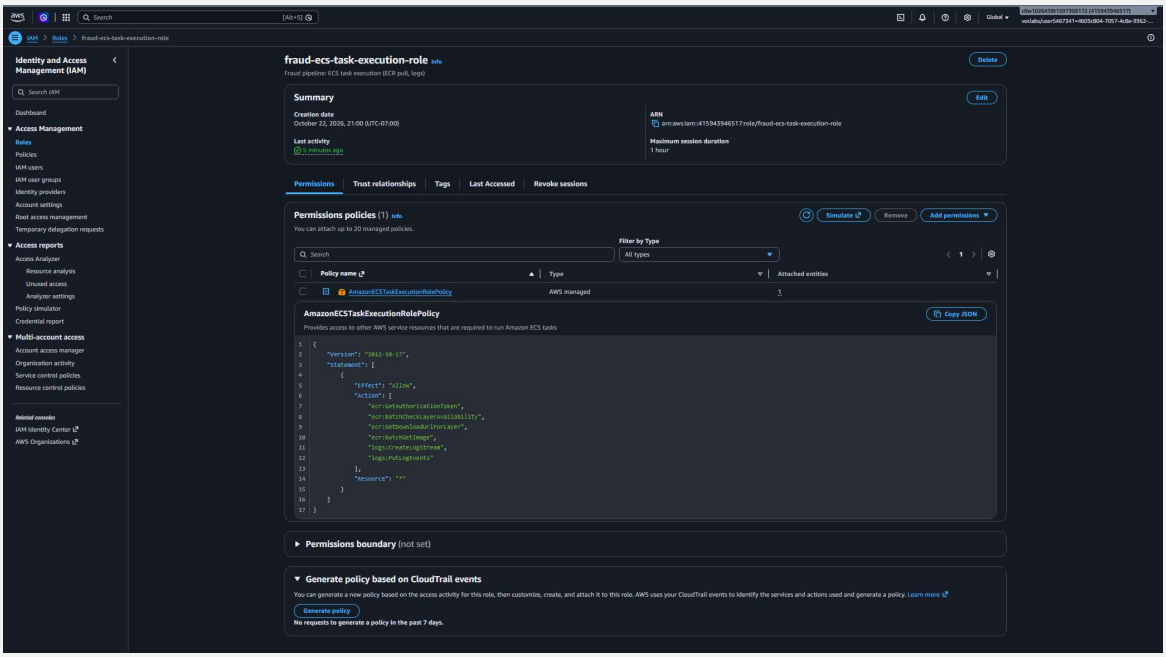
| Service or component | IAM role or service role used | What does this role allow the service to do? |
|---|---|---|
| AWS Glue training job and crawler | fraud-glue-training-role (trusted by glue.amazonaws.com) | Run Glue jobs and crawlers and write their logs to CloudWatch, through the AWS managed policy AWSGlueServiceRole. Read the training data (data/*) and the job script (scripts/*) in the project bucket. Read, write and delete only under models/ and glue-temp/, where it saves the trained model as fraud_detection_model_latest plus a dated archive copy. It has no access to other buckets or services. |
| AWS Lambda retraining function (fraud-model-refresh) | fraud-refresh-lambda-role (trusted by lambda.amazonaws.com) | Write invocation logs to CloudWatch, through AWSLambdaBasicExecutionRole. Read and delete new files in data/incoming/ and write them to data/archive/. Start only the fraud-detection-training Glue job (glue:StartJobRun). Redeploy only the fraud-detection-api ECS service (ecs:UpdateService). |
| Amazon ECS agent launching the API containers | fraud-ecs-task-execution-role (trusted by ecs-tasks.amazonaws.com) | Pull the API image from Amazon ECR and send container logs to the /ecs/fraud-detection-api CloudWatch log group, through AmazonECSTaskExecutionRolePolicy. ECS itself uses this role; the application code never sees it. |
| Prediction API container (FastAPI on Fargate) | fraud-ecs-task-role (trusted by ecs-tasks.amazonaws.com) | Read-only access to the trained model: list and download objects under models/ in the project bucket. ECS gives the container temporary credentials for this role automatically, so no access keys are stored in the code, image or environment variables. |
| Amazon S3 → Lambda trigger | Lambda resource-based policy (statement s3-new-data) | Allow s3.amazonaws.com to invoke fraud-model-refresh, but only for events from the project bucket in this account. |
| Amazon EventBridge → Lambda trigger | Lambda resource-based policy (statement glue-succeeded) | Allow events.amazonaws.com to invoke fraud-model-refresh, but only from the fraud-training-succeeded rule, which matches successful runs of the training job. |
| ECS service, load balancer, auto scaling | AWS service-linked roles: AWSServiceRoleForECS, AWSServiceRoleForElasticLoadBalancing, AWSServiceRoleForApplicationAutoScaling_ECSService | Roles that AWS creates and manages for its own services. They let ECS create network interfaces for Fargate tasks and register them with the load balancer's target group, let the load balancer manage its network resources, and let Application Auto Scaling change the service's task count based on CPU. |



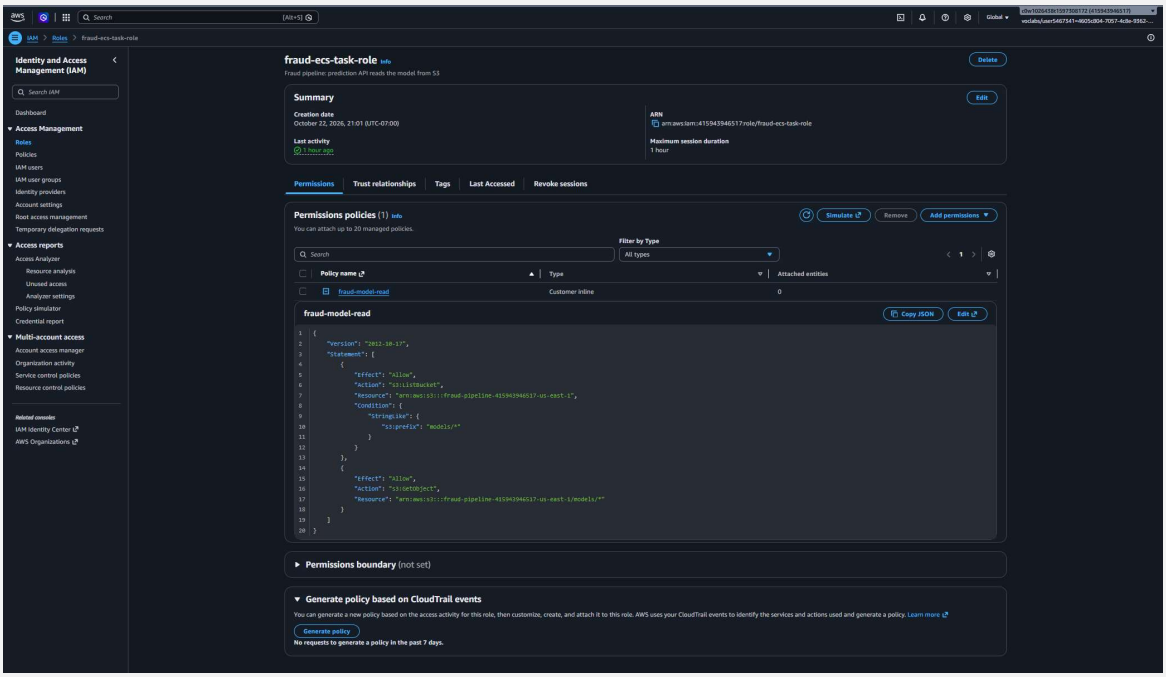

Caption: Glue role, with the full fraud-bucket-access policy



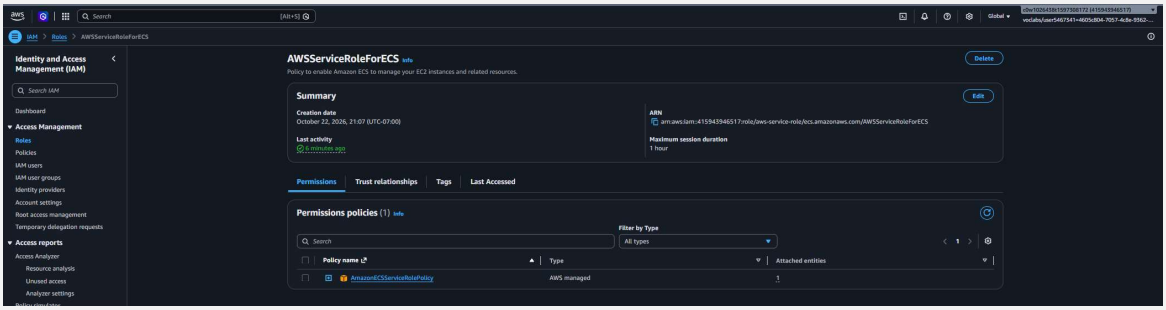

Caption: Lambda role, with the full fraud-refresh-access policy



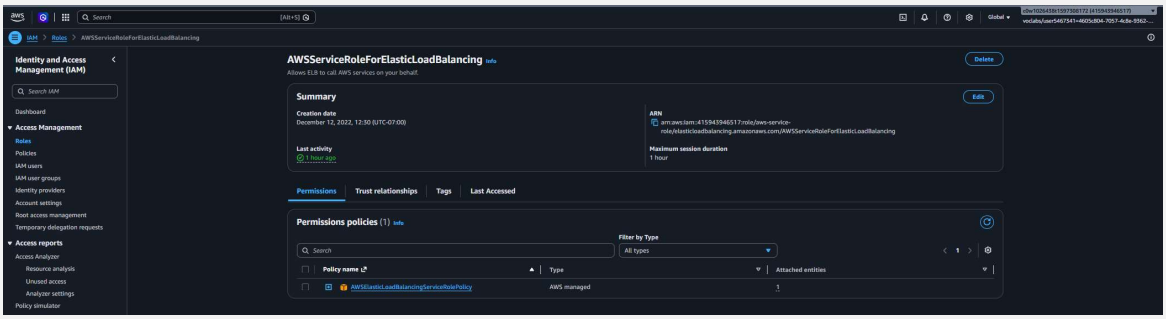

Caption: ECS task execution role, with AmazonECSTaskExecutionRolePolicy expanded



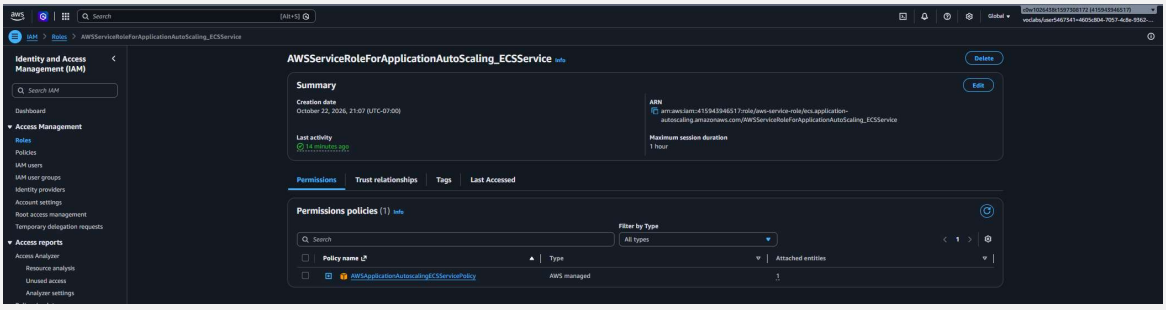

Caption: API task role, with the full fraud-model-read policy



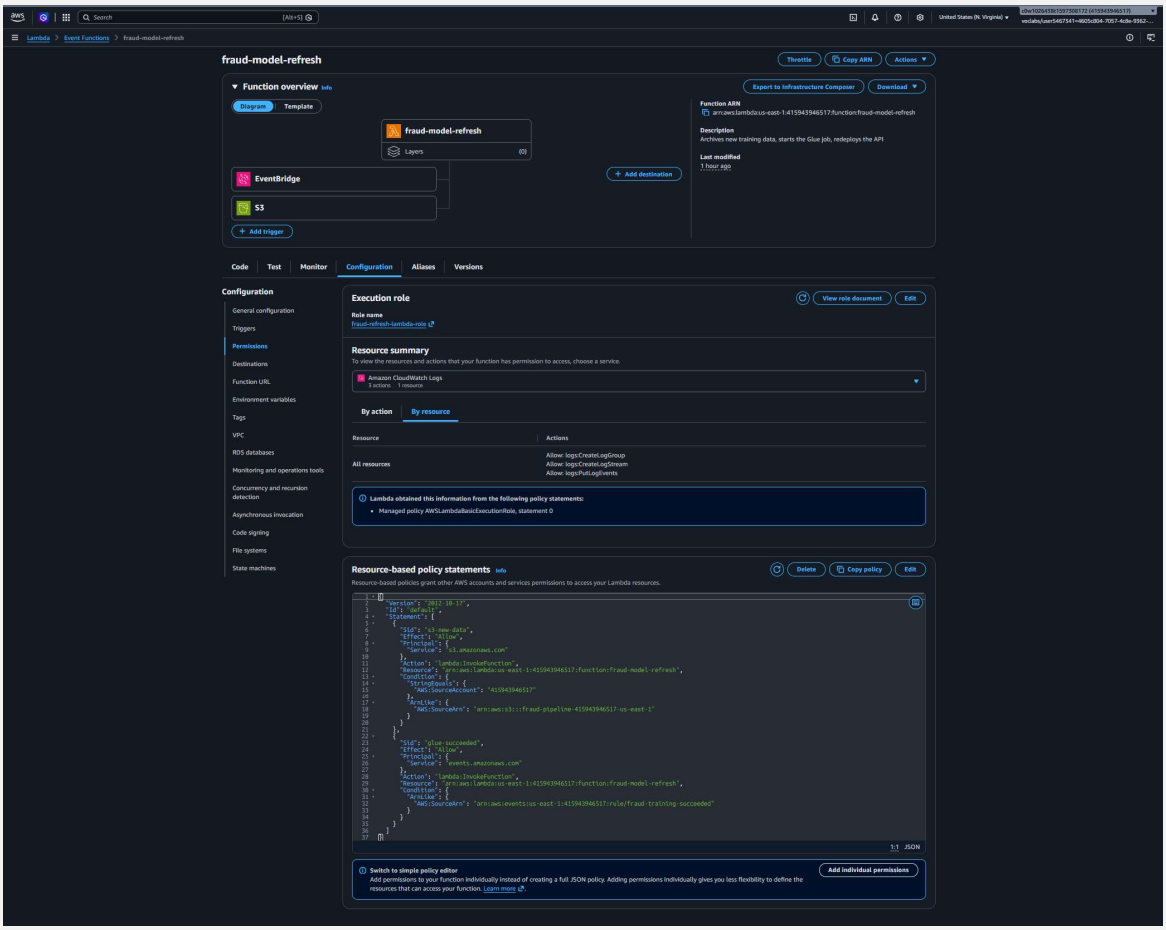

Caption: ECS service-linked role



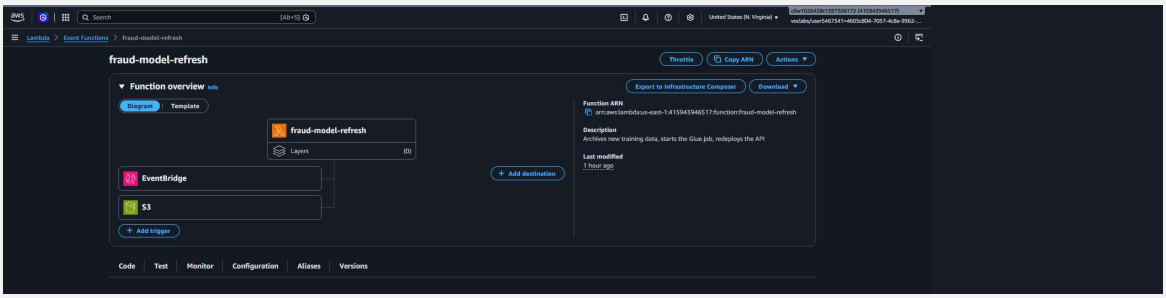

Caption: Load balancer service-linked role




Caption: Auto scaling service-linked role




Caption: Lambda Configuration > Permissions: the execution role, and both resource-based statements (s3-new-data, glue-succeeded)




Caption: the Lambda overview with S3 and EventBridge wired in as triggers, which also works as retraining evidence

## Final Workflow Explanation

| Question | Answer |
|---|---|
| Where is input data stored? | In the Amazon S3 bucket fraud-pipeline-415943946517-us-east-1. The original training data is under data/raw/creditcard.csv. New data is uploaded to data/incoming/, and processed files are moved to data/archive/ after retraining starts. |
| How does the training job run? | As a serverless AWS Glue 5.0 Spark job (fraud-detection-training) on two G.1X workers. It reads the CSV from S3 (DATA_URL), prepares the features, and trains the Random Forest pipeline. It runs on demand or when the retraining Lambda starts it, and bills only while it runs (about 4 minutes per run). |
| Where is the trained model saved? | In the same S3 bucket. Each run writes the model to models/fraud_detection_model_latest/ and a dated copy to models/archive/fraud_detection_model_<timestamp>/, so earlier models can be recovered. |
| How does the API access the trained model? | When an API container starts on ECS Fargate, it downloads models/fraud_detection_model_latest/ from S3 to /tmp and loads it with Spark. Access comes from the ECS task role fraud-ecs-task-role, which can only read models/, so no credentials are stored in the code or image. |
| How is retraining triggered? | Uploading a new .csv to data/incoming/ fires an S3 event that invokes the fraud-model-refresh Lambda. The Lambda archives the file and starts the Glue job on it. When training succeeds, an EventBridge rule invokes the Lambda again, which redeploys the ECS service so the API loads the new model. |